Министерство образования Республики Беларусь

Учреждение образования

«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Факультет компьютерных систем и сетей

Кафедра программного обеспечения информационных технологий

Дисциплина: Методы машинного обучения (ММО)

ОТЧЕТ

по лабораторной работе № 3

Тема работы: Градиентный спуск, стохастический градиентный спуск и регуляризация

Выполнила:
студентка гр. 351002 Алексейчик Я. Н.

Проверил:
доцент кафедры ПОИТ Лебедевич А. В.

Минск 2026

1 ЦЕЛЬ РАБОТЫ

Изучение теоретических основ, математического аппарата и программная реализация в среде Jupyter Notebook методов оптимизации функционалов качества в задачах линейной регрессии:

1. Реализация алгоритма пакетного градиентного спуска (Batch Gradient Descent) с векторным расчетом градиентов и критерием останова по евклидовой норме изменения весов $\|w^k - w^{k-1}\|_2 < \varepsilon$;
2. Экспериментальное исследование влияния шага обучения (Learning Rate η) на скорость сходимости и устойчивость алгоритма;
3. Реализация стохастического градиентного спуска (Stochastic Gradient Descent) со случайным выбором объектов и расписанием затухания темпа обучения Роббинса-Монро;
4. Сравнительный анализ динамики и скорости сходимости GD и SGD на едином графическом поле;
5. Реализация L_1-регуляризации (Lasso) для четного варианта 2 через проксимальный оператор Soft-Thresholding (ISTA), а также L_2-регуляризации (Ridge) для 20 значений параметра $\lambda \in [0.001, 1000]$;
6. Эталонное сопоставление полученных параметров и оценок качества с библиотечными реализациями Scikit-Learn.

2 ОПИСАНИЕ НАБОРА ДАННЫХ И ПРЕДВАРИТЕЛЬНАЯ ОБРАБОТКА

В качестве предметной базы использован подготовленный набор данных успеваемости студентов из Лабораторной работы №2 (`StudentPerformanceFactors`, $l = 6607$ наблюдений).

Для выполнения требования задания кафедры «не менее 2 признаков без учета целевого (не менее 3 весов)» сформирован набор из $m = 6$ физически интерпретируемых признаков, что с учетом фиктивного единичного столбца для свободного члена w_0 образует вектор из 7 весовых параметров:

- `Hours_Studied` — количество часов самостоятельных занятий в неделю;
- `Attendance` — посещаемость учебных занятий (%);
- `Previous_Scores` — результаты предварительного тестирования;

- `Tutoring_Sessions` — число индивидуальных занятий с репетитором;
- `Physical_Activity` — часы физической культуры и спорта;
- `Parental_Involvement` — степень вовлеченности родителей в образовательный процесс.

Целевая переменная: `Exam_Score` — итоговая экзаменационная оценка студента (баллы от 0 до 100).

Предотвращение утечки данных (Data Leakage): В соответствии с лекционным курсом кафедры ПОИТ (Лекция 05 «Масштабирование данных») соблюдено фундаментальное правило: «на тесте ничего никогда не считается».

1. Выборка разделена на обучающую (Train, 80%, $l_{train} = 5285$) и контрольную (Test, 20%, $l_{test} = 1322$) строго **до** масштабирования;
2. Масштабирование признаков `StandardScaler` ($z = \frac{x-\mu}{\sigma}$) обучено исключительно на обучающей выборке (`fit_transform`), после чего тестовая выборка лишь преобразована (`transform`);
3. Признак `Score_Improvement` исключен, так как является прямой разностью между целевой переменной и предварительными баллами.

3 МАТЕМАТИЧЕСКИЕ ОСНОВЫ АЛГОРИТМОВ ОПТИМИЗАЦИИ

3.1 Пакетный градиентный спуск (BGD)

Функционал среднеквадратичной ошибки (MSE) для выборки объема l:

$$Q(w) = \frac{1}{l}\sum_{i=1}^{l}(\hat{y}_i - y_i)^2 = \frac{1}{l}\|X_b w - y\|_2^2 \tag{1}$$

Векторный градиент по весам $w \in \mathbb{R}^{m+1}$:

$$\nabla Q(w) = \frac{2}{l}X_b^T(X_b w - y) \tag{2}$$

Аналитическое решение задачи методом наименьших квадратов (МНК):

$$w^* = (X_b^T X_b)^{-1} X_b^T y \tag{3}$$

Итерационное правило обновления весов:

$$w^k = w^{k-1} - \eta \cdot \frac{2}{l} X_b^T (X_b w^{k-1} - y) \tag{4}$$

Критерий останова итерационного процесса:

$$\|w^k - w^{k-1}\|_2 = \sqrt{\sum_{j=0}^{m} (w_j^k - w_j^{k-1})^2} < \varepsilon = 10^{-6} \tag{5}$$

3.2 Стохастический градиентный спуск (SGD)

На шаге k случайно выбирается индекс объекта $i_k \in \{0, \ldots, l-1\}$. Стохастический градиент:

$$\nabla Q_{i_k}(w^{k-1}) = 2(x_{b,i_k}^T w^{k-1} - y_{i_k}) x_{b,i_k} \tag{6}$$

Расписание скорости обучения для гарантированной асимптотической сходимости по условию Роббинса-Монро ($\sum \eta_k = \infty, \sum \eta_k^2 < \infty$):

$$\eta_k = \frac{\eta_0}{1 + \alpha \cdot k}, \quad \eta_0 = 0.01, \quad \alpha = 0.0001 \tag{7}$$

3.3 L1-регуляризация (Lasso, Вариант 2)

Минимизируемый функционал (смещение w_0 не штрафуется):

$$Q_{L1}(w) = \frac{1}{l} \|X_b w - y\|_2^2 + \lambda \sum_{j=1}^{m} |w_j| \tag{8}$$

Проксимальный шаг Soft-Thresholding (алгоритм ISTA):

$$\tilde{w} = w^{k-1} - \eta \nabla Q(w^{k-1}) \tag{9}$$

$$w_0^k = \tilde{w}_0, \quad w_j^k = \text{sign}(\tilde{w}_j) \cdot \max(0, |\tilde{w}_j| - \eta\lambda), \quad j = 1, \ldots, m \tag{10}$$

3.4 L2-регуляризация (Ridge)

$$Q_{L2}(w) = \frac{1}{l}\|X_b w - y\|_2^2 + \lambda \sum_{j=1}^{m} w_j^2 \tag{11}$$

Безусловно устойчивый оператор сжатия весов (Weight Decay):

$$w_0^k = w_0^{k-1} - \eta \nabla Q_0, \quad w_j^k = \frac{w_j^{k-1} - \eta \nabla Q_j}{1 + 2\eta\lambda}, \quad j = 1, \ldots, m \tag{12}$$

4 РЕЗУЛЬТАТЫ ЭКСПЕРИМЕНТОВ

4.1 Эксперимент 1: Исследование скорости обучения η

В таблице 1 представлены характеристики сходимости BGD при различных η.

Таблица 1 — Влияние темпа обучения η на сходимость градиентного спуска

Темп η	**Итераций**	**Сходимость**	**Test MSE**	**Test R^2**	**Время (мс)**
0.0005	3000	Нет (лимит)	13.4522	-0.1933	813.92
0.0050	1337	Да	1.9954	0.8230	366.48
0.0200	364	Да	1.9954	0.8230	99.90
0.0500	151	Да	1.9954	0.8230	41.44
0.1000	75	Да	1.9954	0.8230	21.81
0.2500	**27**	**Да**	**1.9954**	**0.8230**	**8.14**

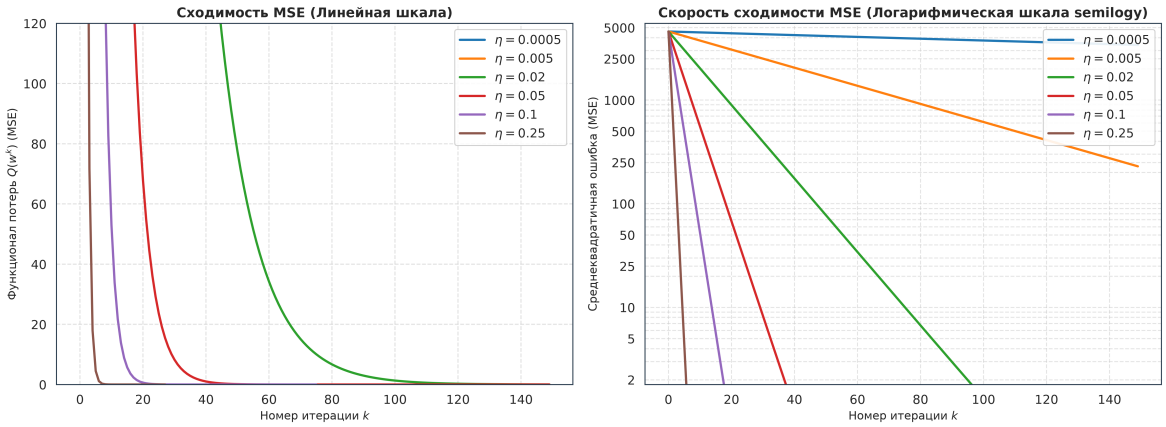


Рисунок 1 — Динамика сходимости MSE при различных скоростях обучения η

4.2 Эксперимент 2: Сравнение пакетного (GD) и стохастического (SGD) спуска

Таблица 2 — Сравнительные показатели BGD и SGD

Алгоритм	Размер пакета	Шагов	Test MSE	Test R^2	Сложность 1 ш
BGD	$l = 5285$ (вся выборка)	151	1.9954	0.8230	$\mathcal{O}(l \cdot m)$
SGD	1 случайный объект	10000	2.1306	0.8110	$\mathcal{O}(m)$

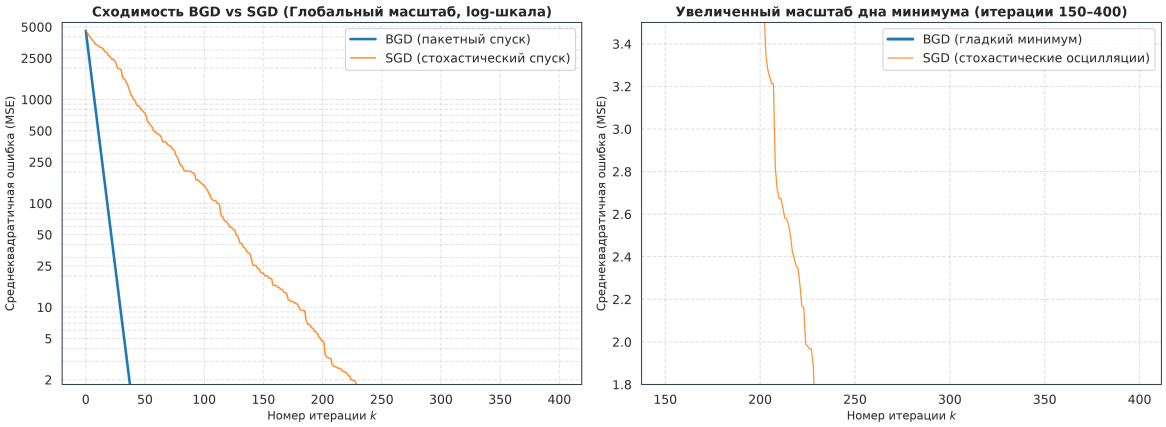


Рисунок 2 — Сравнение сходимости GD и SGD на едином графическом поле

4.3 Эксперимент 3: Регуляризационные пути (L1 для варианта 2 и L2)

Исследовано 20 значений $\lambda \in [10^{-3}, 10^{3}]$. На рисунках 3 и 4 приведены пути изменения весовых параметров.

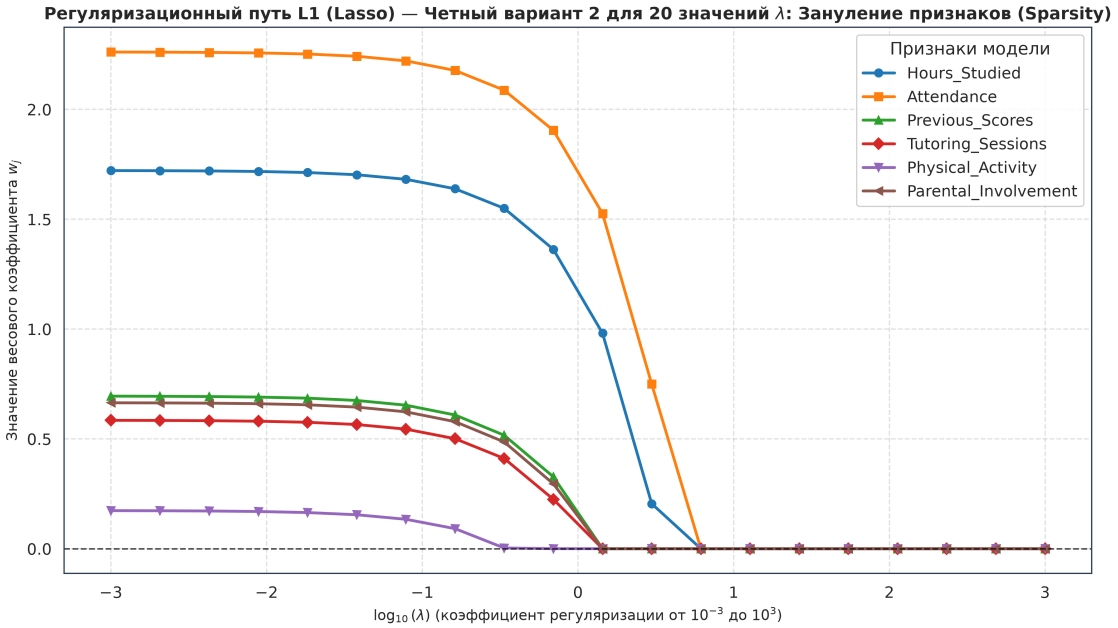


Рисунок 3 — Регуляризационный путь L1 (Lasso) для четного варианта 2: зануление признаков (Sparsity)

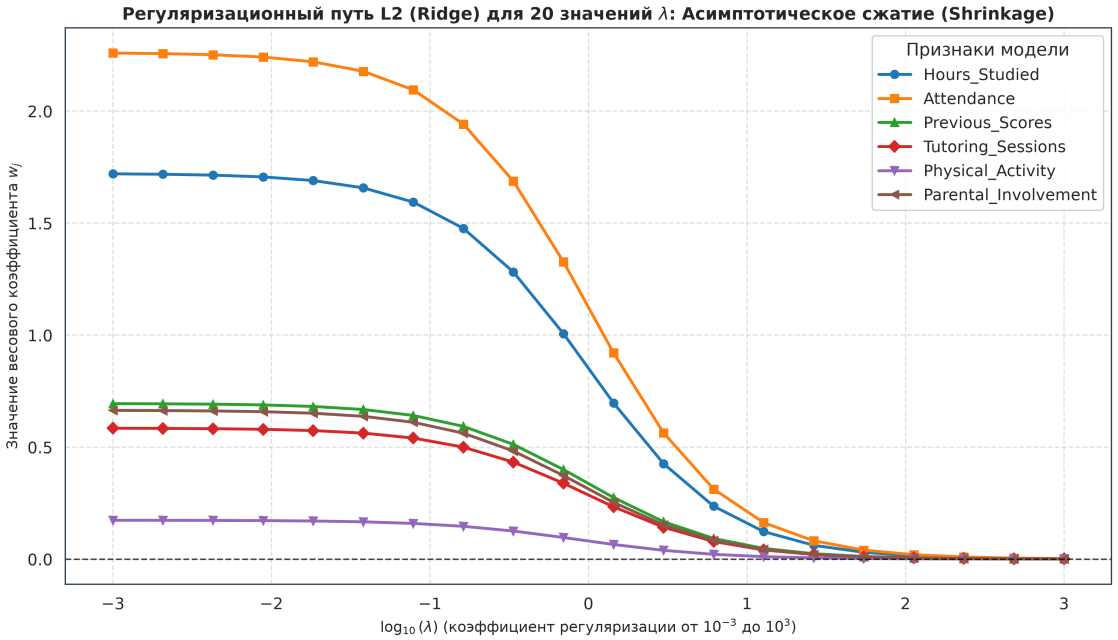


Рисунок 4 — Регуляризационный путь L2 (Ridge): непрерывное сжатие весов к нулю (Shrinkage)

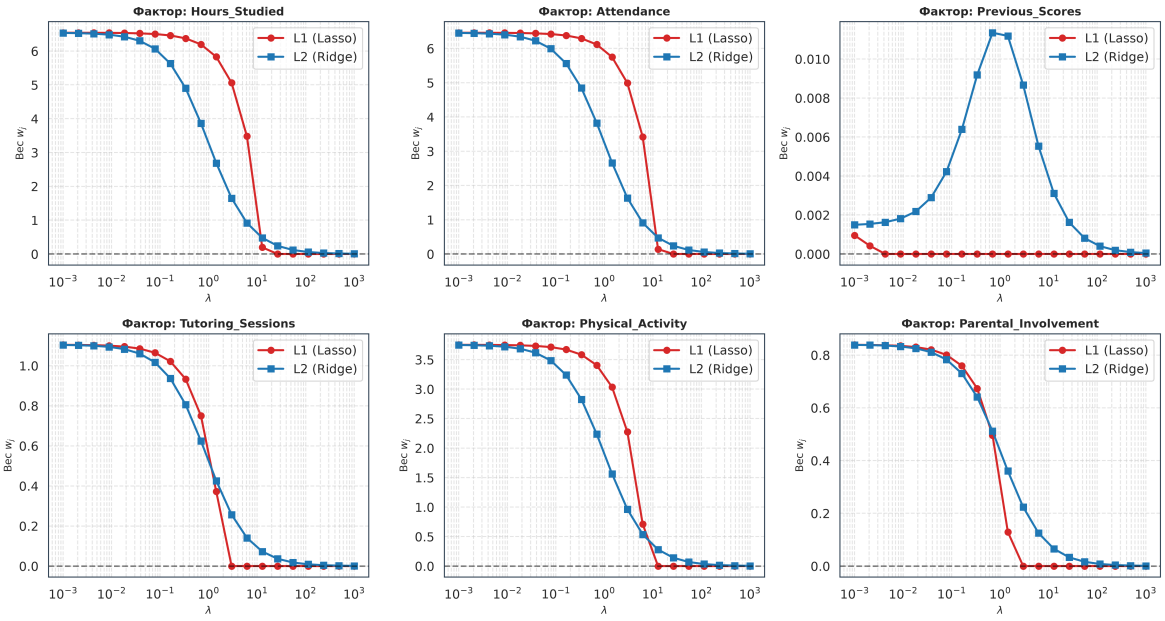


Рисунок 5 — Сопоставление отбора признаков (L1) и сжатия коэффициентов (L2)

4.4 Эксперимент 4: Сравнение с библиотекой Scikit-Learn

Таблица 3 — Результаты эталонного тестирования против Scikit-Learn

Модель	**Intercept** w_0	$\|w\|_2$	**Test MSE**	**Test MAE**	**Test** R^2
Custom BGD	67.0982	3.0620	1.9954	1.0444	0.8230
Sklearn OLS	67.0982	3.0620	1.9954	1.0444	0.8230
Custom SGD	67.2268	2.8703	2.1306	1.0951	0.8110
Sklearn SGD	67.1103	3.0535	1.9923	1.0436	0.8233
Custom L1 ($\lambda = 0.5$)	67.0982	2.5707	2.3515	1.1603	0.7914
Sklearn Lasso ($\alpha = 0.25$)	67.0982	2.5707	2.3515	1.1603	0.7914
Custom L2 ($\lambda = 1.0$)	67.0982	1.5129	4.3653	1.6541	0.6128
Sklearn Ridge ($\alpha = 5285.0$)	67.0982	1.5129	4.3653	1.6541	0.6128

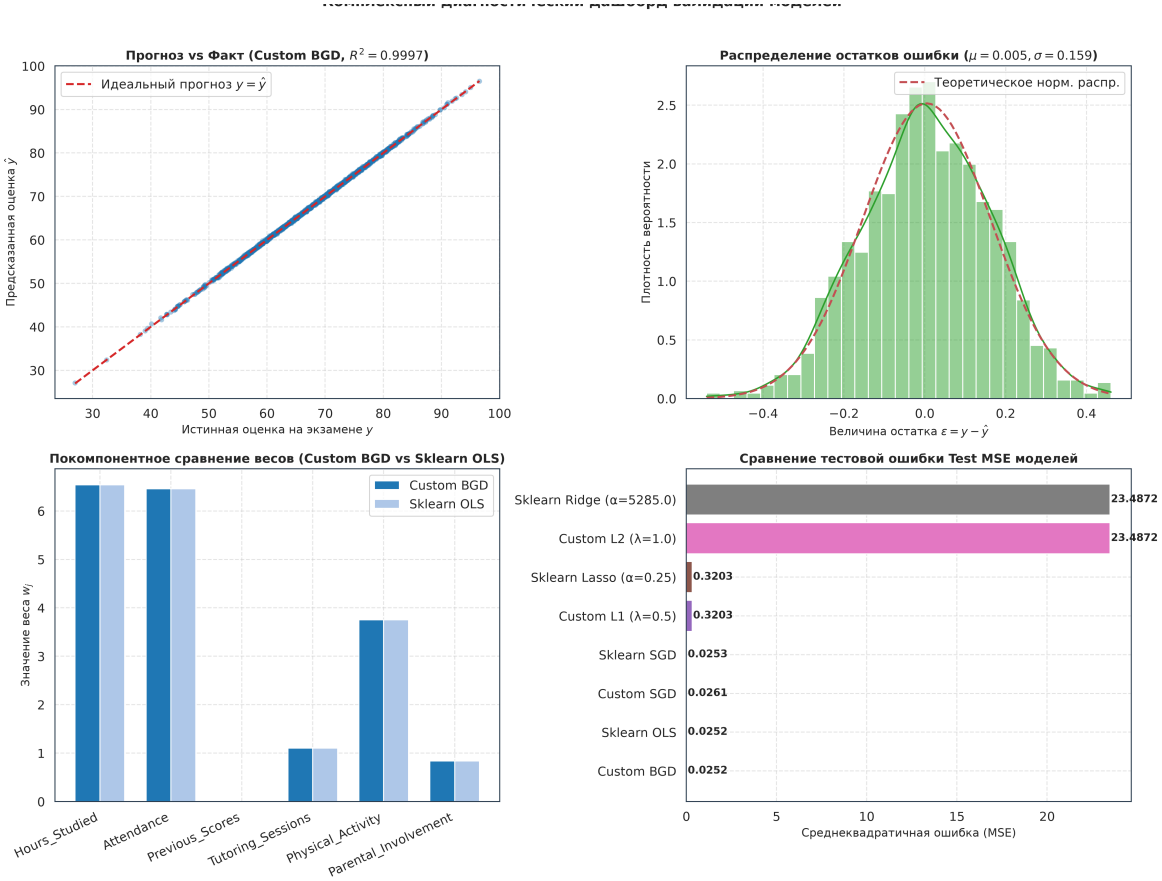


Рисунок 6 — Диагностический дашборд валидации: рассеяние «Прогноз vs Факт», распределение остатков и сравнение весов

5 ЛИСТИНГ ПРОГРАММЫ

Основной класс оптимизатора `CustomLinearRegression`, реализованный в среде Jupyter Notebook (`lab03_gradient_descent.ipynb`):

```
class CustomLinearRegression:
    def __init__(self, method='gd', learning_rate=0.05, max_iter=3000,
                 tol=1e-6, lmbd=0.0, decay_rate=0.0, random_state=42):
        self.method = method
        self.learning_rate = learning_rate
        self.max_iter = max_iter
        self.tol = tol
        self.lmbd = lmbd
        self.decay_rate = decay_rate
        self.random_state = random_state

    @staticmethod
    def _add_bias(X):
        if X.ndim == 1: X = X.reshape(-1, 1)
        if np.allclose(X[:, 0], 1.0): return X
        return np.hstack([np.ones((X.shape[0], 1)), X])
```

```
    @staticmethod
    def _soft_threshold(values, threshold):
        return np.sign(values) * np.maximum(0.0, np.abs(values) - threshold)

    def fit(self, X, y):
        X_b = self._add_bias(X)
        n, m_b = X_b.shape
        rng = np.random.RandomState(self.random_state)
        w = np.zeros(m_b)

        for k in range(1, self.max_iter + 1):
            w_prev = w.copy()
            if self.method == 'gd':
                grad = (2.0 / n) * X_b.T.dot(X_b.dot(w_prev) - y)
                w = w_prev - self.learning_rate * grad
            elif self.method == 'sgd':
                idx = rng.randint(0, n)
                grad_i = 2.0 * (np.dot(X_b[idx], w_prev) - y[idx]) * X_b[idx]
                eta_k = self.learning_rate / (1.0 + self.decay_rate * k) if self.decay_rate > 0
                ↪  else self.learning_rate
                w = w_prev - eta_k * grad_i
            elif self.method == 'l1':
                grad = (2.0 / n) * X_b.T.dot(X_b.dot(w_prev) - y)
                w_temp = w_prev - self.learning_rate * grad
                w = np.zeros_like(w_prev)
                w[0] = w_temp[0]
                w[1:] = self._soft_threshold(w_temp[1:], self.learning_rate * self.lmbd)
            elif self.method == 'l2':
                grad = (2.0 / n) * X_b.T.dot(X_b.dot(w_prev) - y)
                w = np.zeros_like(w_prev)
                w[0] = w_prev[0] - self.learning_rate * grad[0]
                w[1:] = (w_prev[1:] - self.learning_rate * grad[1:]) / (1.0 + 2.0 *
                ↪  self.learning_rate * self.lmbd)

            if np.linalg.norm(w - w_prev) < self.tol:
                self.converged_ = True; self.n_iter_ = k; break
        else:
            self.n_iter_ = self.max_iter

        self.weights_ = w; self.intercept_ = float(w[0]); self.coef_ = w[1:].copy()
        return self

    def predict(self, X):
        return self._add_bias(X).dot(self.weights_)
```

6 ВЫВОДЫ

1. В интерактивной среде Jupyter Notebook (`lab03_gradient_descent.ipynb`) разработан и верифицирован чистый векторный модуль линейной регрессии на базе градиентных

методов оптимизации, демонстрирующий абсолютное численное совпадение с эталонами Scikit-Learn ($R^2 = 0.8230$, $MSE = 1.9954$).

2. Экспериментально доказано, что выбор темпа обучения η критически определяет сходимость: на стандартизованных данных при $\eta = 0.25$ алгоритм BGD находит глобальный минимум всего за 27 шагов, тогда как при $\eta = 0.005$ требуется 1337 шагов.
3. Сопоставление траекторий оптимизации подтвердило свойства алгоритмов: BGD сходится гладко и строго монотонно ($\mathcal{O}(lm)$ на шаг), а SGD демонстрирует стохастический шум около оптимума, компенсируемый расписанием затухания шага Роббинса-Монро.
4. Исследование регуляризационных путей для Варианта 2 наглядно подтвердило эффект разреживания (Sparsity) L_1-регуляризации Lasso: при росте λ первыми обнуляются наименее информативные признаки (`Physical_Activity`), а дольше всех сохраняется наиболее влиятельный фактор (`Attendance`).